Roteiro

- **Pilhas**
  - Conceito
  - Exemplos de uso
  - Criando a estrutura da pilha
  - Inserindo célula
  - Consultando as células
  - Removendo a célula
  - Esvaziando a Pilha

Pilha

LIFO (ou FILO)

Last In, First Out

Último a entrar, primeiro a sair

Operações Principais

- Insere-se elementos no **topo** da pilha
- Remove-se apenas o elemento que estiver no **topo**



Analogia: Uma pilha de panquecas exemplifica a estrutura LIFO.

Exemplos de Uso de Pilha

code

Compiladores

Gerenciamento de funções recursivas e escopo de variáveis.

undo

Editores de Texto

Implementação do mecanismo de "Desfazer" e "Refazer" (Undo/Redo).

public

Navegação Web

Histórico de navegação para voltar às páginas anteriores visitadas.

A estrutura de Pilha (LIFO) é fundamental para processos que requerem retorno ao estado anterior.

Criando a Estrutura da Pilha

1. Estrutura do Elemento (Nó)

```
typedef struct reg {
   int conteudo;
   struct reg *prox;
} tcelula;
```

Estrutura que representa cada elemento da pilha.

- **conteudo:** Armazena o valor do dado.
- ***prox:** Ponteiro para o próximo elemento da pilha.

2. Estrutura Auxiliar de Controle das Operações da Pilha

```
typedef struct pilha {
   tcelula *topo;
} tpilha;
```

Facilita as operações de inserção e remoção.

- ***topo:** Aponta para o topo da pilha.

Inserir Elemento em Pilha

```
void insere(int x, tpilha *pi)
{
   tcelula *novo = (tcelula*) malloc(sizeof(tcelula));
   novo->conteudo = x;
   novo->prox = pi->topo;
   pi->topo = novo;
}
```

1. Alocação e Atribuição

Cria-se um novo elemento na memória e atribui-se o valor de 'x' ao seu conteúdo.

2. Encadeamento

O novo elemento aponta para o elemento que era, até então, o topo da pilha.

3. Atualização do Topo

O ponteiro de controle da pilha passa a apontar para o novo elemento criado.

Imprimindo Toda a Pilha

```
void imprime(tpilha *pi)
{
   if (pi->topo == NULL)
   {
      printf("\nPilha Vazia\n");
   }
   else
   {
      tcelula *aux = pi->topo;
      printf("\n[");
      do
      {
         printf(" %d\n", aux->conteudo);
         aux = aux->prox;
      } while (aux != NULL);
      printf("]\n");
   }
}
```

1. Verificação de Vazio

O algoritmo primeiro checa se a pilha está vazia (topo == NULL) antes de tentar percorrer.

2. Percurso com Auxiliar

Utiliza-se um ponteiro auxiliar para percorrer os nós do topo até o fim, sem perder a referência da pilha.

3. Exibição Circular

O laço 'do-while' garante a impressão de cada elemento até que o ponteiro chegue ao final (NULL).

Removendo Elemento da Pilha

```
void remove(tpilha *pi)
{
   if (pi->topo == NULL)
   {
      printf("\nPilha Vazia\n");
   }
   else
   {
      tcelula *aux = pi->topo;
      printf("%d removido", pi->topo->conteudo);
      pi->topo = pi->topo->prox;
      free(aux);
   }
}
```

1. Verificação de Segurança

Antes de remover, o código verifica se a pilha contém elementos (topo != NULL).

2. Desconexão do Topo

O ponteiro 'aux' guarda o topo atual, e o topo da pilha é movido para o próximo elemento.

3. Liberação de Memória

A função 'free(aux)' desaloca o espaço de memória do elemento removido da pilha.

Esvaziando a Pilha

```
void esvazia(tpilha *pi)
{
   if (pi->topo == NULL)
   {
      printf("\nPilha Vazia\n");
   }
   else
   {
      tcelula *aux = pi->topo;
      do
      {
         pi->topo = pi->topo->prox;
         free(aux);
         aux = pi->topo;
      } while (aux != NULL);
   }
}
```

1. Verificação de Estado

O código inicia conferindo se a pilha já está vazia para evitar operações desnecessárias.

2. Remoção Iterativa

Através de um laço 'do-while', cada elemento é desconectado e o topo é atualizado sequencialmente.

3. Limpeza de Memória

Para cada nó removido, a função 'free' é chamada para liberar o espaço alocado dinamicamente.





Ciência da Computação

Estrutura de Dados

Pilha

Aldo Moura
aldo.moura@souunit.com.br